

INTERNAL — LEARNING DOCUMENT

# ExitAI Codebase Explained

A plain-English walkthrough of every part of the project — written for someone new to coding who built it with AI and now needs to understand and present it.

Covers: what it does · the tech stack · coding basics · every folder and key file · how agents work · how RAG works · the database · manager Q&A.

Generated: 20 September 2026 | ExitAI project — Exit AI/

## Table of Contents

| # | Section                                        |
|---|------------------------------------------------|
| 1 | What Is This Project — plain English           |
| 2 | The Big Picture — tech stack in beginner terms |
| 3 | Coding Basics — words you'll see everywhere    |
| 4 | Folder-by-Folder Tour                          |
| 5 | File-by-File Explanations (all key files)      |
| 6 | How the Agents Work — step by step             |
| 7 | How RAG Works — the AI assistant explained     |
| 8 | The Database — tables and security             |
| 9 | Likely Manager Questions + Honest Answers      |

SECTION 1

# What Is This Project?

When someone resigns from a company, it normally involves weeks of back-and-forth emails, missed checklists, forgotten laptop returns, and HR staff manually chasing people across departments. Things fall through the cracks. The person leaves without a relieving letter. IT forgets to revoke their system access. Finance settles the wrong amount.

**ExitAI** is a software platform that manages this entire process — called *employee offboarding* — using artificial intelligence. It replaces the email chains and paper checklists with a structured, automated system that everyone can see in real time.

## The Problem It Solves

**Without ExitAI**

- HR manually tracks each exit in spreadsheets
- IT is notified late about access revocation
- Finance doesn't know when dues are settled
- Managers miss knowledge-transfer deadlines
- No central view of where each exit stands

**With ExitAI**

- AI automatically generates a checklist on resignation
- Every department gets notified and has their own dashboard
- Risk score is calculated automatically (HR-only)
- AI reviews exit interview responses for themes
- Everything tracked in one place, in real time

## Who Uses It?

| Role     | What They Do on ExitAI                                                                       |
|----------|----------------------------------------------------------------------------------------------|
| Employee | Submits resignation, fills exit interview, uploads documents, tracks their own exit timeline |
| Manager  | Reviews and approves knowledge-transfer tasks, sees their team's exits                       |
| HR       | Oversees all exits, sees risk scores, reads AI-generated insights, issues relieving letters  |
| IT       | Handles deprovisioning (revoking access, collecting laptop, disabling accounts)              |
| Finance  | Confirms final dues are settled before the employee is released                              |

One sentence summary:

ExitAI is an AI-powered employee exit management platform with five role-based dashboards, an AI assistant, and automated agents that generate checklists, assess risk, and coordinate every department — all stored securely in a cloud database.

## SECTION 2

## The Big Picture — Tech Stack in Beginner Terms

### What is a Frontend?

The **frontend** is everything you see in the browser — the dashboards, buttons, forms, and tables. It runs on your computer (inside the browser tab), not on any server. In ExitAI, the frontend is built with **React** (a JavaScript library for building web screens) and **Vite** (a tool that bundles all the code together so the browser can load it fast). We chose React because it breaks the UI into small reusable pieces called *components*, which makes it much easier to manage a complex app with five different dashboards.

### What is a Backend?

The **backend** is code that runs on a server — not in your browser. It handles things the browser shouldn't do directly: writing to the database, sending emails, running the AI agents. In ExitAI, the backend has two parts: **Supabase Edge Functions** (small serverless functions that run in Supabase's cloud, written in TypeScript/JavaScript) and a **Python agent service** that runs locally on the developer's machine and orchestrates all the AI agents.

### What is a Database?

A **database** is like a giant set of organized spreadsheets that can be searched very quickly. ExitAI uses **Supabase**, which gives us a PostgreSQL database (a popular, powerful type of database), user authentication, file storage, and serverless functions — all in one service. Every exit case, task, employee profile, and AI output is stored as rows in database tables.

### What is an LLM?

An **LLM (Large Language Model)** is the AI that reads and writes text — the same kind of AI behind ChatGPT or Claude. ExitAI uses LLMs to generate exit checklists, review knowledge-transfer documents, analyse exit interviews, and produce risk narratives. The LLM is not running on the developer's computer — it is accessed via an API call to a gateway called **Portkey**, which currently routes requests to an Azure-hosted AI model using the Anthropic API format.

### What is RAG?

**RAG (Retrieval-Augmented Generation)** is a technique that lets the AI answer questions based on your own documents rather than just its general training. Instead of hoping the AI already knows your company's exit policy, you feed it the actual policy document. ExitAI uses RAG to power the "Ask a

question" assistant — employees and HR can ask about exit procedures and get answers grounded in the real policy. (Explained in detail in Section 7.)

## What is an Agent?

An **agent** is an AI-powered program that takes a task, decides what steps to take, calls tools (like querying the database or asking the LLM), and produces a result. ExitAI has a system of agents built with **LangGraph** — a framework for wiring agents together. There is one central *supervisor* agent that coordinates several specialist agents (HR, IT, Finance, Risk, etc.). This is called a **hub-and-spoke architecture**: the supervisor (hub) routes work to the specialist agents (spokes), and the spokes only talk back to the hub, never to each other.

## Stack Summary Table

| Layer              | Tool Used                            | Why This Tool                                                  |
|--------------------|--------------------------------------|----------------------------------------------------------------|
| UI / Frontend      | React + Vite                         | Component-based; fast hot-reload during development            |
| Database + Auth    | Supabase (Postgres)                  | Managed DB + row-level security + file storage in one service  |
| Edge Functions     | Supabase Edge Functions (TypeScript) | Serverless — no server to manage; runs in Supabase's cloud     |
| Agent Framework    | LangGraph (Python)                   | Lets you wire agents as a graph; supervisor pattern built-in   |
| LLM Access         | Portkey gateway → Azure OpenAI       | One gateway, one set of keys, works with multiple AI providers |
| Vector Store (RAG) | pgvector in Supabase                 | Stores document embeddings in the same DB — no extra service   |
| Icons              | Tabler Icons                         | Free, consistent, SVG icon set that looks clean at all sizes   |

## SECTION 3

## Coding Basics — Words You'll See Everywhere

Before reading the file explanations, learn these building blocks. Every explanation in Section 5 uses these words — once you know them here, you'll recognise them in the code.

### const / let / var — storing a value with a name

These words *create a named box* to hold a value. `const` means the box can never be replaced (most common — use when you know the value won't change). `let` means it can. `var` is the old way — still works but rarely used now.

```
const role = 'hr'           // role is always 'hr', can't change
let count = 0                // count can be updated later
count = count + 1           // now count is 1
```

**Why const is used most:** It makes code easier to read and safer — you know that value will never accidentally be changed somewhere else in the file.

### function — a reusable block that does a job

A function is a named set of instructions you can run over and over. You define it once, call it by name whenever you need it. Functions take *inputs* (called parameters) and can give back an *output* (called return value).

```
function greet(name) {
  return 'Hello, ' + name
}
greet('Siva') // gives back: 'Hello, Siva'
```

**Why it's used:** Instead of writing the same logic 10 times, write it once and call the function 10 times. Easier to fix bugs (fix in one place).

## import / export — bringing code from another file

`export` means "make this available to other files". `import` means "bring that thing in so I can use it here". This is how large projects stay organised — each file does one job, and other files can use it.

```
// In supabase.js — exports the connection
export const supabase = createClient(URL, KEY)

// In App.jsx — imports it to use
import { supabase } from './lib/supabase'
```

**Why it's used:** Keeps files small and focused. A 5,000-line file would be unmanageable — splitting into imports keeps everything findable.

## async / await — waiting for something slow to answer

Some operations take time — loading data from a database, sending an email, calling an AI.

`async` marks a function as "this will do something that takes time". `await` means "pause here and wait for this to finish before moving on". Without `await`, the code would rush ahead before the database replied.

```
async function getEmployee(id) {
  const result = await supabase.from('profiles').select('*').eq('id', id)
  return result.data
}
```

**Why it's used:** Database queries and API calls can take 100ms–2 seconds. `await` prevents the code from moving on before the answer arrives.

## Variables — the basic data types

A **variable** is any named value. Its *type* describes what kind of value it holds:

- **String** — text, wrapped in quotes: `"hello"` or `'Siva'`
- **Number** — a number (no quotes): `42` or `0.75`
- **Boolean** — true or false: `true`, `false`. Used for on/off decisions
- **Array** — a list of values: `["hr", "it", "finance"]` — accessed by position (0, 1, 2...)
- **Object** — a collection of named values: `{ name: "Siva", role: "hr" }`

```
const name = "Aravidhan" // string
const score = 0.82 // number
const isActive = true // boolean
const roles = ['hr', 'it'] // array
const user = { name: 'Siva', role: 'hr' } // object
```

## if / else — making a decision

The computer checks a condition; if it's true it does one thing, otherwise ( `else` ) it does another. This is literally every decision in the code.

```
if (role === 'hr') {
  showAllCases()
} else if (role === 'employee') {
  showOwnExit()
} else {
  showLogin()
}
```

**Why it's used:** Every role sees a different dashboard, every agent takes a different path based on conditions — all of that is `if/else`.

## a "query" — asking the database for data

A query is an instruction to the database: "give me this data". In ExitAI, most queries use Supabase's JavaScript client, which reads like plain English: `.from('exit_tasks')` means "from the exit\_tasks table", `.select('*')` means "give me all columns", `.eq('case_id', id)` means "where case\_id equals this id".

```
supabase.from('exit_tasks')
  .select('*')
  .eq('case_id', caseId)
  .eq('status', 'pending')
```

This asks: "from exit\_tasks, give me all columns where case\_id matches AND status is pending." The database returns a list of matching rows.

## an "API call" / fetch — asking another service for something

An **API** (Application Programming Interface) is a way for two programs to talk to each other. A *fetch* or API call is your code sending a request to another service (like the AI, or Supabase) and getting a response back. Think of it like texting someone and waiting for their reply.

```
const response = await fetch('http://localhost:8787/activate-exit', {
  method: 'POST',
  body: JSON.stringify({ case_id: id })
})
const result = await response.json()
```

Here the frontend is "texting" the Python agent service to start the exit pipeline for a case.

## JSON — a way to format data

**JSON** (JavaScript Object Notation) is a standard text format for passing data between systems. It looks like a JavaScript object — key-value pairs, arrays, nested objects. When the frontend calls an API, it sends and receives JSON.

```
{ "case_id": "abc-123", "status": "in_progress", "risk_score": 0.72 }
```

Almost every API response — from Supabase, from the AI — comes back as JSON. JSON is language-neutral (Python and JavaScript both understand it), which is why it's used for all communication.

## a component — a reusable piece of the UI (React)

In React, a **component** is a function that returns a piece of the screen. You can use it many times with different data — like a LEGO brick. In ExitAI, `Sidebar`, `LoginPage`, and `PageHead` are all components.

```
function TaskCard({ title, status }) {  
  return (  
    <div>  
      <p>{title}</p>  
      <span>{status}</span>  
    </div>  
  )  
}  
// Used like: <TaskCard title="Return laptop" status="pending" />
```

Without components, every page would have hundreds of lines of duplicated HTML. Components let you write a card once and reuse it everywhere.

SECTION 4

# Folder-by-Folder Tour

| Folder / File                    | What It's For                                                                                                                                                                                                                                                                                                                     |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>src/</code>                | <b>All frontend code.</b> Everything the browser runs: dashboards, login, navigation, components. Written in React (JSX files).                                                                                                                                                                                                   |
| <code>src/routes/</code>         | Each role has its own sub-folder: <code>employee/</code> , <code>hr/</code> , <code>manager/</code> , <code>it/</code> , <code>finance/</code> . Each folder has a Layout file (the sidebar + shell) and a Pages file (the actual content pages).                                                                                 |
| <code>src/components/</code>     | Shared building blocks used across dashboards: <code>Sidebar.jsx</code> , <code>LoginPage.jsx</code> , <code>PageHead.jsx</code> .                                                                                                                                                                                                |
| <code>src/lib/</code>            | Utility functions and the Supabase client. Things like <code>supabase.js</code> (the database connection), <code>format.js</code> (date formatting), <code>clearanceStatus.js</code> (logic for checking clearances).                                                                                                             |
| <code>src/styles/</code>         | CSS design tokens — colours, spacing, font sizes. All dashboards share these so they look consistent.                                                                                                                                                                                                                             |
| <code>agents/</code>             | <b>All Python agent code.</b> The AI pipeline that runs when someone submits a resignation. Has its own virtual environment ( <code>.venv/</code> ) and Python packages separate from the JavaScript frontend.                                                                                                                    |
| <code>agents/hub/</code>         | The <b>supervisor</b> — the orchestrator that coordinates all agents. <code>supervisor.py</code> is the main file.                                                                                                                                                                                                                |
| <code>agents/spokes/</code>      | Each specialist agent lives here: <code>hr_agent.py</code> , <code>risk_agent.py</code> , <code>it_deprovisioning_agent.py</code> , <code>finance_agent.py</code> , <code>exit_intel_agent.py</code> , <code>compliance_agent.py</code> , and others.                                                                             |
| <code>agents/core/</code>        | Shared tools all agents use: <code>config.py</code> (secrets/database connection), <code>llm.py</code> (how to call the AI), <code>trace.py</code> (live terminal logging), <code>prompts.py</code> (loads prompt text files), <code>notifications.py</code> (email sending), <code>calendar_booking.py</code> (Google Calendar). |
| <code>agents/service.py</code>   | The HTTP bridge — a small web server (running on your machine on port 8787) that the frontend calls to trigger agent actions.                                                                                                                                                                                                     |
| <code>supabase/</code>           | <b>Database migrations.</b> Each <code>.sql</code> file is a change to the database structure — tables, security rules, views. Run in order (0001, 0002...). These files are the only source of truth for what the database looks like.                                                                                           |
| <code>supabase/functions/</code> | Supabase Edge Functions — serverless code that runs in Supabase's cloud. The <code>ask/</code> function powers the RAG assistant; <code>submit-resignation/</code> creates the exit case when an employee submits.                                                                                                                |
| <code>scripts/</code>            | Utility scripts: <code>seed.js</code> creates 103 demo employee profiles and exit cases; <code>ingest_docs.js</code> loads the exit policy documents into the RAG store;                                                                                                                                                          |

`exit_policy.md` is the actual company policy document.

`docs/`

Documentation files. `ExitAI_Code_Explained.pdf` — this document.

`agents/core/prompts/`

The instructions given to the AI for each task. Stored as markdown `.md` files — e.g. what to write in an exit checklist, how to evaluate a KT document. Loaded at runtime by `prompts.py`.

`.env`

Secret keys — database passwords, AI API keys. **Never committed to git, never visible in the browser.** Listed in `.gitignore`.

## SECTION 5

## File-by-File Explanations

This section walks through every important file — what it does, why it exists, and what the key lines mean in plain English.

Frontend **src/App.jsx**

**What it does:** This is the *root* of the entire web app. It runs first when someone opens the website. It checks whether the user is logged in, finds out their role (employee/HR/manager/IT/finance), and then shows the right dashboard for that role.

**Why it exists:** Without this file, the app would have no starting point — it would not know who is logged in or where to send them.

```
const [session, setSession] = useState(undefined) // undefined = still loading
const [role, setRole] = useState(null) // will be 'hr', 'employee', etc.
```

These two lines use `const` and `useState` (a React tool) to create two boxes: one holds the logged-in session, one holds the user's role. They start empty ( `undefined` / `null` ) and get filled in after the app loads.

```
supabase.auth.getSession().then(({ data }) => setSession(data.session))
```

This line asks Supabase: "is there a logged-in session right now?" When the answer comes back, it saves it in the `session` box. This is an **async API call** to Supabase Auth.

```
supabase.from('profiles').select('role').eq('id', session.user.id)
```

Once logged in, this **database query** asks: "from the profiles table, give me the role column where the id matches this user." That tells the app whether to show the HR dashboard, IT dashboard, etc.

```
if (!session) return <LoginPage />
```

If there's no session (not logged in), show the login screen. This is an **if statement** that renders a React **component**.

**Frontend** `src/routes/AppRoutes.jsx`

**What it does:** Maps web addresses (URLs) to the right page. When you go to `/employee/tasks`, this file shows the Tasks page. When you go to `/hr/all-exits`, it shows the All Exits page for HR.

**Why it exists:** Without routing, the browser would have no idea which page to show for each URL. Every tab on every dashboard corresponds to a route defined here.

```
<Route path="/employee/tasks" element={role === 'employee' ? <Employee.Tasks /> : <Navigate to={`/${
```

This says: "if the URL is `/employee/tasks`, check the role. If the user is an employee, show the Tasks component; otherwise redirect them to their own dashboard." This is how the app prevents a manager from visiting an employee's page.

**Frontend** `src/components/LoginPage.jsx`

**What it does:** Shows the login screen. Handles email + password input and calls Supabase Auth to sign in.

**Why it exists:** Every user must be authenticated before seeing any data. Supabase Auth handles the actual password checking — this component just provides the form.

Frontend

src/routes/employee/EmployeePages.jsx

**What it does:** All the pages an employee sees: their Dashboard, My Exit progress, Tasks list, Documents upload, Exit Interview form, and Timeline of their exit stages.

**Why it exists:** This is the employee's entire experience in the app — the interface through which they submit their resignation and track their exit.

```
const STAGE_ORDER = ['hr', 'manager', 'it', 'finance']
```

This **array** (a list) defines the order of exit stages. The timeline on the employee's dashboard follows this order — HR comes first, finance last.

```
function stageState(tasks) {
  const escalations = tasks.filter(t => t.title?.startsWith('Escalated'))
  const work = tasks.filter(t => !t.title?.startsWith('Escalated'))
  return {
    blocked: escalations.some(e => (e.escalation_state ?? 'open') === 'open'),
    complete: work.length > 0 && work.every(t => t.status === 'done'),
  }
}
```

This **function** figures out whether a stage (like "Manager & KT") is complete, blocked, or still in progress. It separates escalation tasks (manager rejected something) from normal work tasks, then checks: are all work tasks done? Is there an open escalation? The result drives the colour of the timeline dot (green = done, red = blocked, etc.).

**Frontend** `src/routes/hr/HrPages.jsx`

**What it does:** The most complex frontend file — all pages for the HR dashboard: All Exits, Escalations, Risk & Compliance, Exit Interviews, Trends, Clearances, Agent Activity, and more.

**Why it exists:** HR is the power user of the system — they see everything, including the AI-generated risk scores, interview analysis, and agent activity logs.

```
const RISK_TONE = { low: 't-success', medium: 't-warning', high: 't-danger' }
```

This **object** maps risk level names to CSS colour classes. When the risk level is "high", the badge turns red ( `t-danger` ). This keeps the display logic consistent — change the colour in one place and it updates everywhere.

```
const humanizeStatKey = (key) =>
  STAT_ACRONYMS[key] ?? key.replace(/_/g, ' ').replace(...)
```

This **function** converts database column names (like `avg_completion_rate` ) into readable English ("Avg completion rate"). The AI analytics agents store data in snake\_case; this makes them readable for HR users without requiring the agents to worry about formatting.

**Python / Agents** `agents/core/config.py`

**What it does:** Loads all the secret keys from the `.env` file (passwords, API keys, database URL) and creates the Supabase database connection that every agent uses.

**Why it exists:** Every agent needs the database and the AI. Rather than each agent file reading the `.env` file itself (and potentially reading the wrong value), they all import from this one file. Fix or change a key here, and it updates for all agents automatically.

```
load_dotenv(Path(__file__).resolve().parent.parent.parent / ".env", override=True)
```

This line finds and reads the `.env` file (the project's secret file) from three folders up from this file's location. `override=True` means "use these values even if the computer already has a setting with the same name" — this was needed because an old Windows system variable was causing the wrong gateway URL to be used.

```
db: Client = create_client(SUPABASE_URL, SUPABASE_SERVICE_KEY)
```

This creates the database connection using the **service key** (a key that bypasses all security rules — agents need this to write to tables that regular users can't touch). The `db` variable is then imported by every other agent file.

Python / Agents **agents/core/llm.py**

**What it does:** The single place in the codebase where the AI (LLM) is called. Every agent that needs the AI calls `ask_claude()` or `ask_claude_json()` from here — they never call the AI directly.

**Why it exists:** Centralises all AI communication. If the gateway URL or model name changes, only this file needs updating.

```
resp = requests.post(
    f"{ANTHROPIC_BASE_URL}/v1/messages",
    headers={ "Authorization": f"Bearer {ANTHROPIC_API_KEY}", ... },
    json={ "model": ANTHROPIC_MODEL, "messages": [...] }
)
```

This is the actual **API call** to the AI gateway. It's a plain HTTP POST request (like filling a web form and pressing Submit) sent to the Portkey gateway URL. The gateway translates it and sends it to the Azure AI model. The result comes back as JSON.

```
text_block = next((b for b in body["content"] if b.get("type") == "text"), None)
```

Some AI models return a "thinking" block before the actual text. This line skips those and finds the real text answer. Without this, the code would crash when the model included reasoning steps.

```
def ask_claude_json(system, user, max_tokens=500):
    text = ask_claude(system, user, max_tokens)
    match = re.search(r"\{.*\}", text, re.DOTALL)
    return json.loads(match.group(0))
```

This function asks the AI and then extracts the JSON object from the response. The AI sometimes wraps its JSON in extra text or backticks — `re.search` is a regular expression that finds the `{...}` block regardless.

Python / Agents

agents/core/trace.py

**What it does:** Provides live, real-time logging in the terminal as agents run. Every agent is decorated with `@traced_node`, which prints when the agent starts, what data it received, when the AI was called (and how many tokens it used), what was written to the database, and when the agent finished — all colour-coded and indented so you can see the hub handing off to each spoke.

**Why it exists:** Without this, running the agent pipeline is a black box — you'd submit a case and see nothing until it finished (or crashed). With `trace.py`, you watch every step live in the terminal.

```
def traced_node(name: str):
    def decorator(fn):
        def wrapper(state, *args, **kwargs):
            start = time.perf_counter()
            _emit(_c(f"└─ {name}", "cyan") + "  ▶ start")
            result = fn(state, *args, **kwargs)
            elapsed = time.perf_counter() - start
            _emit(_c(f"└─ {name}", "cyan") + f"  ✓ done {elapsed:.2f}s")
            return result
        return wrapper
    return decorator
```

`traced_node` is a **decorator** — a special function that wraps another function. When you write `@traced_node("HR agent")` above a function, Python automatically wraps it: print "start" when it's called, run the real function, print "done" with timing when it finishes. This is how all 14 agents get logging without any of them having to write their own print statements.

Python / Agents `agents/hub/supervisor.py`

**What it does:** The *orchestrator* — the conductor of the entire agent orchestra. It defines the order in which agents run for an exit case, and handles the conditional path (manager approved vs. manager rejected).

**Why it exists:** Without the supervisor, each agent would have to know about all the others and call them in sequence. Instead, they each do their own job, and the supervisor decides who goes next.

```
# The flow:
# entry -> hr -> manager_gate --[approved]--> it -> compliance -> finance -> assess -> END
#                                     ^--[rejected]--> escalate -> END
```

This comment describes the entire pipeline. After the HR stage, there's a *gate*: if the manager approved the knowledge transfer, the pipeline continues to IT, Compliance, Finance, then the Risk Assessment. If the manager rejected, it branches to Escalation and stops.

```
_graph = StateGraph(SupervisorState)
_graph.add_node("hr", _hr_stage)
_graph.add_node("manager_gate", _manager_gate)
_graph.add_conditional_edges("manager_gate", _route_after_manager,
    {"approved": "it", "rejected": "escalate"})
```

This builds the graph. `StateGraph` is a `LangGraph` object that holds the pipeline. Each `add_node` call registers a stage. `add_conditional_edges` is the decision point — if `_route_after_manager` returns "approved", go to "it"; if "rejected", go to "escalate". The graph is a living diagram you can follow step by step.

```
class SupervisorState(TypedDict):
    case_id: str
    kt_text: str | None
    simulate_rejection: bool
    log: list[str]
```

This is the **state** — the data passed between all agents as the pipeline runs. `case_id` identifies which exit case we're processing. Every agent receives this state, does its work, and returns it (possibly with additions). Think of it as a baton in a relay race.

Python / Agents

agents/spokes/hr\_agent.py

**What it does:** Two jobs: (1) Generate an exit checklist — calls the AI with the employee's role and department to produce a list of HR and manager tasks; (2) Review a knowledge-transfer document — reads the submitted KT doc and asks the AI to identify gaps.

**Why it exists:** The checklist is the very first output of an exit — it populates the task lists that HR, the manager, and IT all use throughout the exit process.

```
@traced_node("HR agent -- generate checklist")
def _generate_checklist(state):
    case = state["case"]
    user = f"Role: {case['role_title']}\nDepartment: {case['department']}"
    state["result"] = ask_claude_json(CHECKLIST_SYSTEM_PROMPT, user)
    return state
```

This function tells the AI: "here's the employee's role and department — generate a checklist." The AI is given a system prompt (instructions about what format and content to produce) and the employee details. The response is a JSON object with HR tasks and manager tasks.

```
IT_OWNED_TITLE_RE = re.compile(
    r"^\s*(?:revoke|de-?provision|disable|deactivate)...", re.IGNORECASE
)
```

This is a *regular expression* — a pattern that matches text. It catches checklist items that belong to IT (like "Revoke access to GitHub") and removes them from the manager's task list. This prevents a manager from seeing tasks they can't perform, which was blocking the manager approval gate.

Python / Agents

agents/spokes/risk\_agent.py

**What it does:** Calculates a risk score for each exit — a number between 0 and 1 that tells HR how much of a risk this departure is to the organisation. Also decides whether the employee is eligible for rehire.

**Why it exists:** HR needs to know which exits are high-risk (e.g. a critical engineer with bad sentiment leaving quickly) so they can prioritise attention.

**Important: No AI is used here.**

The risk score is calculated with pure maths — a weighted formula. The AI is NOT making this judgement. This was a deliberate decision: arithmetic is predictable, auditable, and never hallucinates.

```
WEIGHTS = {"tenure": 0.2, "role": 0.3, "sentiment": 0.3, "tasks": 0.2}
```

The four factors and how much each one counts. Role criticality and interview sentiment each count for 30%; tenure and outstanding tasks each count for 20%. These weights add up to 1.0 (100%).

```
risk_score = round(
    tenure * WEIGHTS["tenure"]
    + role * WEIGHTS["role"]
    + sentiment * WEIGHTS["sentiment"]
    + tasks_risk * WEIGHTS["tasks"],
    3
)
```

The actual calculation. Each factor (a number between 0 and 1) is multiplied by its weight, then they're added together. `round(..., 3)` keeps only 3 decimal places. The result is a single number like `0.72`.

```
risk_level = "high" if risk_score >= 0.66 else "medium" if risk_score >= 0.4 else "low"
```

Converts the number to a label:  $\geq 0.66$  = high risk,  $\geq 0.4$  = medium, below 0.4 = low.

Python / Agents `agents/spokes/exit_intel_agent.py`

**What it does:** Reads an employee's exit interview text and uses the AI to produce: a summary, a sentiment label (positive/neutral/negative), key themes (e.g. "poor management", "better compensation elsewhere"), and a rehire recommendation.

**Why it exists:** HR cannot read every exit interview carefully. This agent distills each interview to a few key facts, and over time the longitudinal mode spots rising themes across all interviews (e.g. "compensation" is trending across Engineering).

```
state["result"] = ask_claude_json(SYSTEM_PROMPT, f"Transcript:\n{state['interview_text']}")
```

The AI gets the system prompt (instructions on what to analyse) plus the interview text. It returns a JSON object with summary, sentiment, themes, and rehire\_eligible fields — all stored in the database for HR to view.

Python / Agents `agents/spokes/it_deprovisioning_agent.py`

**What it does:** Two stages. First, generate a deprovisioning plan (the list of IT tasks: revoke GitHub access, collect laptop, disable SSO account). Second, after the IT team approves each task, "execute" them through a mock adapter and record the audit trail.

**⚠ Honest Note:** The "Execute" step uses a **MockITAdapter** — it simulates revoking access but does not actually connect to Okta, Jamf, GitHub, or any real IT system. No real access is revoked by this code.

```
ASSET_RE = re.compile(r"laptop|macbook|device|headset|card|asset|collect", re.I)
IDENTITY_RE = re.compile(r"sso|identity|account", re.I)
```

These patterns categorise tasks by keyword. A task titled "Collect MacBook" matches `ASSET_RE` and is classified as an asset-return action. A task with "SSO" is an identity action. The mock adapter uses this classification to simulate the right type of action.

Python / Agents `agents/spokes/finance_agent.py`

**What it does:** Checks whether finance has been cleared for an exit. Requires two things: all other stages done (HR, Manager, IT tasks all complete) AND the Finance dashboard shows dues marked as settled. Only then marks the finance task as done.

**Why it exists:** The relieving letter cannot be issued until finance confirms the employee owes nothing. This agent enforces that gate programmatically.

**No LLM here either.**

Finance clearance is a simple checklist: are all prior tasks done? Is the dues flag set? Pure logic, no AI.

Python / Agents `agents/service.py`

**What it does:** A small web server (HTTP server) running on your local machine on port 8787. The browser cannot directly run Python — so this service acts as a bridge. The React frontend sends requests to this service, which then runs the Python agent pipeline.

**Why it exists:** Supabase Edge Functions run in Supabase's cloud and cannot call Python code on your laptop. The frontend can call localhost, so this service is the connection point between the browser and the Python agents.

```
routes = {
    "/activate-exit": lambda body: activate_case(body["case_id"]),
    "/submit-exit-interview": lambda body: submit_exit_interview(body["case_id"]),
    "/manager-approve": lambda body: manager_approve(body["case_id"]),
    ...
}
```

This **object** maps URL paths to Python functions. When the frontend calls `POST /activate-exit`, the service runs `activate_case()`, which generates the HR checklist and emails the manager. Each route triggers a specific agent action.

```
# ponytail: stdlib http.server, not Flask/FastAPI -- neither is a project
# dependency and this is one route.
```

The comment explains why plain Python's built-in web server was used instead of a popular framework like Flask. Flask would be unnecessary weight for a single-machine development server with 9 routes.

## Database supabase/migrations/0001\_schema.sql

**What it does:** Creates all the main database tables: `profiles`, `exit_cases`, `exit_tasks`, `exit_interviews`, `exit_docs`, and `trend_alerts`.

**Why it exists:** A database has no tables until you create them. This file is run once (by Supabase) to set up the entire structure the app relies on.

```
create table if not exists exit_cases (
  id          uuid primary key default gen_random_uuid(),
  status      text not null default 'open' check (status in ('open', 'in_progress', 'complete')),
  risk_level  text check (risk_level in ('low', 'medium', 'high')),
  ...
);
```

This creates the `exit_cases` table. Each column has a type ( `uuid`, `text`, `numeric` ) and optional constraints ( `not null` means this column must always have a value, `check` means only these specific values are allowed). The `risk_level` column can only hold "low", "medium", or "high" — the database rejects anything else.

## Database supabase/migrations/0002\_rls.sql

**What it does:** Implements Row Level Security (RLS) — the database security model. Defines which roles can read or write which rows. This is why an employee cannot see their own risk score, and a manager cannot see another manager's exits.

**Why it exists:** The UI hides sensitive columns, but that's not enough — a determined user could bypass the UI and query the database directly. RLS enforces the rules at the database level: even if you bypass the app, you see 0 rows for tables you're not allowed to access.

```
-- exit_cases: ONE policy. HR only. No employee policy, no manager policy.
create policy exit_cases_hr_select on public.exit_cases
  for select to authenticated
  using (public.app_current_role() = 'hr');
```

Only HR can select (read) rows from the `exit_cases` table directly. Employees and managers have no policy on this table — they get 0 rows no matter what they query. Their only view of cases is through restricted database views that hide the risk and assessment columns.

**Scripts** `scripts/ingest_docs.js`

**What it does:** Reads the exit policy document ( `scripts/exit_policy.md` ), chops it into small chunks (pieces of text), converts each chunk into an embedding (a list of numbers representing its meaning), and saves both the text and the numbers into the `exit_docs` database table. This is the RAG setup step.

**Why it exists:** The AI cannot "read" a document — it needs the document converted into a mathematical format (embeddings) that it can search. Run this script once to load the policy; after that the RAG assistant can answer questions about it.

**Scripts** `scripts/seed/seed.js`

**What it does:** Creates 103 fake employee profiles (Emp001–Emp100 plus named demo accounts for HR, Manager, IT, Finance) and 15 demo exit cases, all with realistic data. This populates the database so there's something to demonstrate.

**Why it exists:** Without seed data, the dashboards would be empty. The seed provides a realistic demo environment for presentations and testing.

**⚠ Honest Note:** All 103 profiles are fake. No real employee data is in this system. Passwords are deterministic and demo-only (e.g. `Emp001@`) — never use this pattern in production.

## SECTION 6

## How the Agents Work — Step by Step

### What is an "agent" in plain English?

An agent is a program that uses AI to make decisions and take actions — reading from the database, calling the AI, writing results back, and deciding what to do next. Think of it as a very capable employee who can read instructions, think about them using AI, and fill in forms in the database — all automatically.

ExitAI uses a **hub-and-spoke** pattern. The *Supervisor* is the hub — it decides the order of work. The specialist agents (HR, IT, Finance, Risk, etc.) are the spokes — they each do one specific job and report back to the hub.

### A Full Exit: From Resignation to Relieving Letter

**EMPLOYEE** submits resignation form in browser ↓ **Supabase Edge Function** (submit-resignation/index.ts) – creates exit\_case row in database – sets status = 'open' ↓ calls localhost:8787/activate-exit **Agent Service** (agents/service.py) – receives the case\_id ↓ **HR Agent** (hr\_agent.py) ← **AUTOMATIC** – asks AI: generate a checklist for this role/department – AI returns list of tasks (HR tasks + manager/KT tasks) – tasks are saved to exit\_tasks table in database – sends email reminder to employee + manager about KT deadlines – books Google Calendar events for KT meetings – case status flipped to 'in\_progress' ↓ pipeline pauses – waits for human action **MANAGER** reviews KT tasks in Manager dashboard ← **HUMAN** – marks each KT task as done (or rejects with reason) – if rejected → Escalation agent creates escalation row → HR sees it – if all done → manager clicks "Approve" ↓ frontend calls /manager-approve **IT Agent** (it\_deprovisioning\_agent.py) ← **AUTOMATIC** – asks AI: generate IT deprovisioning plan for this employee – creates IT tasks in database (revoke access, collect laptop, etc.) ↓ waits for IT team action **IT TEAM** approves each task in IT dashboard ← **HUMAN** – clicks Approve on each task – MockITAdapter "executes" the action and records audit log ↓ **Compliance Agent** (compliance\_agent.py) ← **AUTOMATIC** – checks: manager approved? IT tasks done? Documents validated? – updates compliance status ↓ **Finance Agent** (finance\_agent.py) ← **AUTOMATIC** – checks: all prior stages done? Dues settled flag set? – marks finance task done if cleared ↓ waits for Finance action **FINANCE TEAM** marks dues settled in Finance dashboard ← **HUMAN** ↓ **Risk Agent** (risk\_agent.py) ← **AUTOMATIC** – calculates risk score (formula, no AI) – writes risk\_level, risk\_score, rehire\_eligible to exit\_cases – HR can now see these fields (not visible to employee/manager) ↓ **HR** issues Relieving Letter from HR dashboard ← **HUMAN** – clicks "Issue Relieving Letter" – frontend updates database, calls /issue-relieving-letter – Email sent to employee with relieving notice – Case status → 'completed' ↓ **EMPLOYEE** sees "Relieving letter issued" in their dashboard

### Automatic Steps

- Checklist generation (AI)
- KT reminder emails + calendar booking
- IT deprovisioning plan (AI)
- IT task execution (mock)
- Compliance check
- Finance clearance check
- Risk score calculation (formula)
- Exit interview analysis (AI)

### Human Steps

- Employee submits resignation form
- Employee fills exit interview
- Employee uploads documents
- Manager approves/rejects KT tasks
- IT team approves each deprovisioning task
- Finance marks dues settled
- HR reviews + issues relieving letter

## What is LangGraph?

LangGraph is the framework that lets you connect agents into a graph (a flowchart). Each agent is a *node*. Edges connect nodes and control the order. *Conditional edges* let you branch: if the manager approved, go to IT; if rejected, go to escalation. The `supervisor.py` file builds this graph in about 20 lines of code — the graph handles all the routing automatically after that.

## SECTION 7

# How RAG Works — The AI Assistant Explained

## What is RAG?

**RAG = Retrieval-Augmented Generation.** In plain English: instead of asking the AI to answer from its general knowledge (which might be wrong or outdated), you first search your own documents for the relevant pieces, then give those pieces to the AI along with the question. The AI answers based on your actual content.

## Step-by-Step Diagram

— SETUP (done once, by running scripts/ingest\_docs.js) — **exit\_policy.md** (the real company exit policy document) ↓ split into chunks **Chunk 1:** "An employee must return all company assets within 3 days..." **Chunk 2:** "Final settlement is processed within 30 days of last working day..." **Chunk 3:** "Knowledge transfer must be completed before the exit date..." ↓ each chunk → OpenAI Embedding API **Embedding:** [0.12, -0.34, 0.87, ... 1536 numbers] (meaning as maths) ↓ saved to database **exit\_docs table:** source | section | content | embedding(vector) — chunk text and its 1536-number vector stored together — QUERY (every time someone asks the assistant a question) — **USER QUESTION:** "How long do I have to return my laptop?" ↓ → OpenAI Embedding API **Question Embedding:** [0.11, -0.33, 0.89, ...] (same format) ↓ cosine similarity search in exit\_docs **match\_exit\_docs()** (SQL function in 0003\_rag.sql) — compares question vector to all chunk vectors — returns 4 most similar chunks ↓ **Top match:** "An employee must return all company assets within 3 days..." ↓ question + matching chunks sent to AI **AI (Claude / Azure OpenAI via Portkey)** — "Based on the exit policy: you have 3 days to return your laptop..." ↓ **ANSWER shown to user**

## The Real Files

| What                                     | Where in the Code                                                                   |
|------------------------------------------|-------------------------------------------------------------------------------------|
| Policy document (source)                 | <code>scripts/exit_policy.md</code>                                                 |
| Ingestion script (runs once)             | <code>scripts/ingest_docs.js</code>                                                 |
| Where chunks+embeddings are stored       | <code>exit_docs</code> table in Supabase (created in <code>0001_schema.sql</code> ) |
| The search function                      | <code>match_exit_docs()</code> in <code>0003_rag.sql</code>                         |
| The Edge Function that answers questions | <code>supabase/functions/ask/index.ts</code>                                        |
| Embedding model used                     | OpenAI <code>text-embedding-3-small</code> (1536 dimensions)                        |
| Index type for fast search               | HNSW index on the vector column (fast approximate nearest-neighbour search)         |

## What are "embeddings" and "cosine similarity"?

An **embedding** is a list of 1,536 numbers that represents the *meaning* of a piece of text — not just its exact words. Two sentences that mean the same thing (even with different words) will have similar numbers. **Cosine similarity** is a way of measuring how close two lists of numbers are. A similarity of 1.0 = identical meaning, 0.0 = completely unrelated. The `match_exit_docs` function finds the chunks with highest similarity to the question — those are the most likely to contain the answer.

### Why this matters for a demo:

The RAG assistant only knows what was ingested. If the exit policy says "3 days for laptop return", the assistant says "3 days". If you change the policy document and re-ingest, the assistant's answers change automatically.

## SECTION 8

## The Database — Tables and Security

### Main Tables (Plain English)

| Table                  | One row = ...                                            | Key columns                                                                                              | Who sees it                                                                                |
|------------------------|----------------------------------------------------------|----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>profiles</b>        | One person<br>(employee, HR,<br>manager, IT,<br>finance) | role, full_name, email,<br>employee_id, department                                                       | Your own row: everyone. All<br>rows: HR only                                               |
| <b>exit_cases</b>      | One exit case<br>(one resignation)                       | employee_name, department,<br>status, last_working_day,<br>risk_level*, risk_score*,<br>rehire_eligible* | * = HR only.<br>Managers/employees access<br>safe columns only via<br>restricted views     |
| <b>exit_tasks</b>      | One checklist<br>item                                    | case_id, stage<br>(hr/manager/it/finance), title,<br>status (pending/done), due_date                     | Everyone sees tasks for their<br>stage. Employees see their<br>own only                    |
| <b>exit_interviews</b> | One exit<br>interview<br>submission                      | summary, sentiment, themes,<br>rehire_eligible, rehire_reason                                            | HR only (the AI-generated<br>fields are sensitive<br>assessment data)                      |
| <b>exit_docs</b>       | One chunk of a<br>policy document<br>+ its embedding     | source, section, content,<br>embedding (1536 numbers)                                                    | Only accessible via the<br>service-key /ask Edge<br>Function — not the browser<br>directly |
| <b>agent_runs</b>      | One agent<br>activity log entry                          | case_id, stage, detail, created_at                                                                       | HR (agent activity dashboard)                                                              |
| <b>kt_reviews</b>      | One KT<br>document review<br>result (AI output)          | summary, gaps, complete                                                                                  | HR and manager only                                                                        |
| <b>trend_alerts</b>    | One rising theme<br>across exit<br>interviews            | theme, department, severity                                                                              | HR only                                                                                    |

### Row Level Security — Why an Employee Can't See Their Own Risk Score

The risk score ( `risk_level` , `risk_score` , `rehire_eligible` ) lives in the `exit_cases` table alongside safe columns like the employee's name and last working day. So why can't the employee just query the table and see their own risk score?

**Because of Row Level Security (RLS)** — a PostgreSQL feature that filters rows *inside the database*, before any data is returned.

Employee logs in → queries `exit_cases` table directly ↓ RLS checks: what policy exists for this user? `exit_cases_hr_select` policy: only if role = 'hr' ↓ employee's role is 'employee', not 'hr'  
**Result: 0 rows returned.** Even though their case is there. ↓ employee's only path to their case data is through: `employee_exit_view` (a restricted database view) — deliberately excludes `risk_level`, `risk_score`, `rehire_eligible` — only exposes: `status`, `last_working_day`, `department`, `role_title`

The views ( `employee_exit_view` , `manager_case_view` , `it_task_view` , `finance_case_view` ) are like windows into the table — each window only shows the columns that role is allowed to see. They run as `security_invoker = false` (the view has permission to see past the RLS rules on the base table, but it explicitly selects a safe column list, so no sensitive data leaks through).

#### Non-negotiable:

These five views must NEVER be changed to `security_invoker = on` . That setting breaks them — the view would be blocked by the same HR-only RLS that protects the table, returning 0 rows for every non-HR user. This has accidentally broken every dashboard three times already.

## SECTION 9

## Likely Manager Questions + Simple Honest Answers

These are questions a manager, director, or technical reviewer might ask in a demo or presentation, with answers you can say out loud.

### Q1: What is RAG, in one sentence?

RAG means the AI searches your own company documents for the relevant part of the policy before answering — so it answers from the real policy, not from general internet knowledge.

### Q2: Is this "real AI" or just pre-written answers?

It's real AI. The exit checklist, the KT document review, and the exit interview analysis are all generated live by an LLM (a language model — the same technology as ChatGPT). Every checklist is freshly generated for each employee's actual role and department. The risk score is not AI — it's a fixed weighted formula — which is intentional, so it's predictable and auditable.

### Q3: What did you build vs. what was automated/mock?

#### **Built and real:**

The five dashboards (employee/HR/manager/IT/finance), the database schema, all security rules (RLS), the AI checklist/KT/interview agents, the risk scoring, the RAG assistant, Google Calendar booking, email logging via Gmail SMTP.

#### **Partially real:**

Emails are sent via Gmail SMTP — but only when the `EMAIL_TEST_RECIPIENT` environment variable is set. Without it, emails are logged to the terminal but not delivered.

#### **Mock / simulated:**

The IT deprovisioning "execution" step uses a MockITAdapter — it records the action in the audit log but does not actually connect to Okta, Jamf, or any real identity provider. No real employee access is revoked by this system.

### Q4: Where is the data stored? Is it secure?

All data is stored in  
**Supabase**

— a managed cloud database (hosted by Supabase on AWS). Authentication is handled by Supabase Auth (industry standard). Access is enforced by PostgreSQL Row Level Security — the database itself enforces who can see what, not just the application code. All secrets (API keys, database password) are stored in a `.env` file that is never committed to git.

#### Q5: What is the tech stack? Is it industry-standard?

Yes. React (frontend) is one of the most widely used UI libraries in the world. Supabase/PostgreSQL is a mature, widely used database platform. LangGraph is a well-established Python framework for building agent pipelines. The LLM access uses the Anthropic API format (the same format used by Claude). Python is the dominant language for AI/ML work.

#### Q6: How does the system know which dashboard to show each person?

When a user logs in, the app reads their role from the `profiles` table (set when the account was created). The role — employee, hr, manager, it, or finance — determines which dashboard they see and which routes they can access. If you try to visit a different role's URL, you are redirected to your own dashboard.

#### Q7: What happens if the manager rejects the knowledge transfer?

An escalation record is created in the database. HR can see it in their Escalations page. HR can either re-route it back to the manager with instructions, or resolve it themselves. The exit pipeline stops at the manager gate — IT and finance agents do not run until the escalation is resolved and the manager approves.

#### Q8: Can the employee see their own risk score?

No. The risk score ( `risk_level` , `risk_score` , `rehire_eligible` ) is an HR-only field. It is enforced at the database level — even if an employee queries the database directly, they get 0 rows from the `exit_cases` table because Row Level Security blocks them. Their only view of their case is through a restricted view that does not include those columns.

#### Q9: How many exit cases can this handle at once?

##### **Current state (development):**

The Python agent service runs on a single machine. It can handle concurrent requests (the HTTP server uses threading), but for high volume it would need to be deployed to a cloud server or containerised (e.g. Docker on AWS/GCP). The database (Supabase/Postgres) can handle

thousands of concurrent cases without changes. The current setup is sufficient for a pilot or department-level rollout.

#### Q10: How do you restart or re-run the agents if something fails?

Each agent is designed to be idempotent — running it twice produces the same result as running it once (it checks if data already exists before inserting). You can re-run any individual agent or the full pipeline for a case with: `python -m agents.run_case <case_id>`. The terminal shows a live trace of everything that ran.

#### Q11: What is a "migration" and why are there 30 of them?

A database migration is a versioned change to the database structure — adding a column, creating a table, adding a security rule. They are numbered (0001, 0002...) and run in order. Starting with 30 migrations is normal for a project that evolved over time — it means the database was built incrementally as features were added, which is the correct approach (versus changing the database in an ad-hoc way with no history).

#### Q12: What is the Portkey gateway?

Portkey is an LLM gateway — a service that sits between our code and the actual AI model. We send requests to Portkey in the Anthropic API format, and Portkey translates them and sends them to whichever AI model is configured (currently Azure OpenAI). This means if we want to switch AI providers, we change one configuration in Portkey, not all the code.

#### Q13: How long does the full exit pipeline take to run automatically?

The automatic steps (checklist generation, interview analysis, risk scoring) each take 2–10 seconds, depending on the AI response time. The full automated pipeline (not counting human approval steps) takes about 30–60 seconds to complete. Human steps (manager approval, IT approvals, finance confirmation) are the main time-consuming steps in a real exit — those take hours or days, not seconds.

#### Q14: What would need to happen to use this in production?

##### Remaining for production readiness:

1. Replace the MockITAdapter with real integrations (Okta for SSO, Jamf for device management, GitHub API for repo access).
2. Deploy the Python agent service to a cloud server (currently runs only on the developer's laptop).

3. Real employee data needs to be loaded (and demo data removed).
4. Email delivery needs the `EMAIL_TEST_RECIPIENT` flag removed and proper email templates designed.
5. A full end-to-end browser review of all five dashboards across all user journeys.
6. Legal/compliance review of what data is stored (risk scores, rehire flags) and data retention policies.

#### Q15: What is the difference between the anon key and the service key?

Supabase gives you two API keys. The **anon key**

is safe to use in the browser — it is subject to Row Level Security (the database filters what you can see based on who you are). The

**service key**

bypasses all RLS and can read and write everything — it is used only in the Python agent service and Edge Functions on the server side, never in the browser. Exposing the service key in the frontend would be a critical security hole.

#### Prepared by:

ExitAI project, September 2026. All code inspected from the actual repository — no assumptions. For technical questions, run `python -m agents.run_case <case_id>` from the project root with the agents virtual environment active to see the live agent trace.